🚀 Lecture 04.07: Tuples & Structural Memory Compactness

📌 Executive Architectural Summary

A tuple is an **immutable, ordered sequence of heterogeneous object references** (`PyTupleObject`). Because tuples cannot change size after allocation, CPython optimizes their memory footprint, eliminating the over-allocation overhead of dynamic lists.

🔬 Core Engineering Mechanics

1. Memory Efficiency: Tuple vs List

- **List Overhead:** Requires tracking current size, allocated capacity, and spare headroom.
- **Tuple Overhead:** Allocated exactly to fit N pointer slots (`ob_item[N]`). No spare headroom.
- **Garbage Collection:** CPython maintains a free list of small tuples, making tuple allocation faster than list allocation.

2. Hashability & Dictionary Keys

An object is hashable if its hash value never changes during its lifetime. Because tuples are immutable, a tuple containing only immutable objects is **hashable** and can serve as a dictionary key or set element (unlike lists).

💻 Production Python Implementation

```
import sys

# 1. Memory Comparison: List vs Tuple of identical data
sample_data = (1, 2, 3, 4, 5, 6, 7, 8)
sample_list = [1, 2, 3, 4, 5, 6, 7, 8]

print(f"Tuple Memory Footprint: {sys.getsizeof(sample_data)} bytes")
print(f"List Memory Footprint:  {sys.getsizeof(sample_list)} bytes")

# 2. Hashability Verification
point_tuple = (12.5, 45.8)
```

```
print(f"Tuple Hash: {hash(point_tuple)}")

# Valid Dictionary Key
locations = {point_tuple: "Warehouse Alpha"}
print("Dictionary with Tuple Key:", locations)
```